Universidad de Guadalajara

Centro Universitario de Guadalajara (CUGDL)

Materia: Bases de Datos

Semestre: 3er semestre

Profesor: Marco Antonio Cisneros Guevara

Nombre del proyecto:

GeoEquidad

(Sistema Inteligente de Focalización y Logística para Asistencia Social)

Avance de Base de Datos y Modelos de Regresión

Participantes:

Christian Edén Ureña Vázquez

Tadeo Salvador Hernández Muñoz

Fecha de entrega:

28 de octubre de 2026

1 Objetivo del proyecto

Construir, limpiar y almacenar en un gestor de base de datos relacional (MySQL) un conjunto de datos oficial a nivel municipal sobre pobreza y carencias sociales en México (CONEVAL), como base analítica del sistema GeoEquidad para evaluar cómo las carencias en servicios básicos, alimentación, salud, educación e ingreso explican el porcentaje de población en situación de pobreza mediante modelos de regresión lineal simple y regresión lineal múltiple.

2 Hipótesis

Hipótesis para Regresión Lineal Simple

Existe una relación lineal positiva y estadísticamente significativa entre el porcentaje de población con carencia por calidad y espacios de la vivienda y el porcentaje total de población en situación de pobreza municipal; es decir, a mayor carencia de vivienda en el municipio, mayor es su porcentaje de pobreza total. Se eligió esta carencia porque, excluyendo el ingreso (que es prácticamente la misma variable que la pobreza), es la carencia con mayor correlación con la pobreza entre los municipios de Jalisco ($r = 0.61$).

Hipótesis para Regresión Lineal Múltiple

Al incorporar simultáneamente múltiples indicadores de carencias sociales (educación, salud, seguridad social, vivienda, servicios básicos y alimentación) junto con variables categóricas de clasificación municipal convertidas en variables indicadoras (0 y 1), el modelo de regresión lineal múltiple explica una mayor proporción de la variación (mayor R^2 ajustado) del porcentaje de pobreza municipal que el modelo de regresión lineal simple.

3 Base de datos: Proceso de obtención de la base de datos

Para el desarrollo de este avance del proyecto GeoEquidad se utilizaron datos abiertos oficiales publicados por el Consejo Nacional de Evaluación de la Política de Desarrollo Social (CONEVAL), organismo encargado de la medición de la pobreza y las carencias sociales en México.

El proceso de obtención se realizó siguiendo estos pasos:

1. Acceso al portal oficial del CONEVAL (www.coneval.org.mx) en la sección de Medición de la Pobreza a nivel municipio (véase Figura 1 / Captura 1).
2. Localización del apartado de Datos Abiertos / Anexo Estadístico a nivel municipal, el cual concentra los indicadores socioeconómicos, porcentajes de pobreza y carencias sociales para los municipios de México (incluyendo los 125 municipios del estado de Jalisco) elaborados a partir de información censal del INEGI.
3. Descarga del paquete de datos oficial (`Concentrado_indicadores_de_pobreza_2020.xlsx`, hoja `Concentradomunicipal`) y extracción del archivo original sin realizar ninguna alteración manual en sus filas o columnas.
4. Resguardo del archivo íntegro en el directorio local del proyecto dentro de la carpeta `01_base_original` (véase Figura 2 / Captura 2), garantizando que el archivo fuente permanezca intacto y que todo el procesamiento posterior se ejecute mediante código en Python sobre una copia controlada.

4 Base de datos limpia: Proceso de limpieza y software utilizado

Software y herramientas utilizadas

- **Lenguaje de programación:** Python 3.13.14 ejecutado en un entorno virtual aislado (`venv`) dentro de Visual Studio Code (VS Code).
- **Librerías de procesamiento y conexión:** `pandas3.1.0` (lectura, filtrado, transformación y exportación de tablas), `openpyxl3.1.5` (lectura del libro Excel `.xlsx`), `mysql-connector-python26.10.0` (inserción en base de datos relacional) y `python-dotenv` (lectura segura de credenciales desde el archivo `.env`).
- **Gestor de base de datos:** MySQL Server 8.0.46 y MySQL Workbench (base de datos: `avance_bd`).

Diagnóstico de la base original y proceso de limpieza paso a paso

El archivo original `Concentrado_indicadores_de_pobreza_2020.xlsx` nunca fue alterado; se cargó íntegro en la tabla `datos_originales` (2,493 filas) y el programa `01_limpieza.py` (Anexo A) trabajó sobre una copia en memoria realizando las siguientes operaciones:

1. **Eliminación de columnas vacías e irrelevantes:** El archivo original contenía 146 columnas (incluyendo una primera columna vacía `Col_00` y mediciones de los años 2010

y 2015). Se seleccionaron las 16 columnas correspondientes a la identificación geográfica y a los indicadores oficiales del corte 2020.

2. **Reconstrucción y estandarización de encabezados:** En el Excel original los nombres de las columnas estaban divididos entre las filas 4 y 5 con saltos de línea y llamadas a notas al pie (por ejemplo, "Población 2020* (leer nota al final del cuadro)"). Se renombraron todas las columnas en formato `snake_case` (minúsculas sin acentos ni espacios).
3. **Filtrado de filas no municipales y acotación al estado de Jalisco:** Del total de 2,493 filas del archivo original, se detectaron y eliminaron 24 filas que correspondían a títulos iniciales, renglones en blanco y notas metodológicas al final del archivo (las cuales generaban entre 7 y 23 valores vacíos NaN por columna). Asimismo, a nivel nacional existían 3 municipios con texto no numérico ("n.d."), por lo que la acotación a Jalisco (`clave_entidad='14'`) se realiza antes de convertir a número. Al filtrar los municipios válidos (clave de 5 dígitos) del estado, se obtuvieron los 125 municipios con 0 valores vacíos, 0 valores "n.d.", 0 duplicados y 0 porcentajes fuera del rango $[0, 100]$.
4. **Conversión de tipos de datos:** Las claves geográficas se estandarizaron como cadenas de 2 y 5 dígitos con cero a la izquierda (`CHAR(2)` y `CHAR(5)`); los conteos de personas se convirtieron a enteros (`INT`) y los porcentajes y promedios a números decimales redondeados a dos cifras (`DECIMAL(6,2)` y `DECIMAL(5,2)`).
5. **Creación de variables categóricas "x" y variables indicadoras (0 y 1):** Se generaron dos clasificaciones categóricas para el análisis: `tipo_municipio_poblacion` (`Urbano_Grande` si la población es de 100,000 personas o más; `Semiurbano_Medio` si está entre 15,000 y 99,999; `Rural_Pequeno` si es menor a 15,000) y `nivel_intensidad_carencias` (`Intensidad_Alta` si el promedio de carencias es de 2.30 o más; `Intensidad_Moderada` entre 1.90 y 2.29; `Intensidad_Baja` por debajo de 1.90), junto con sus 4 columnas indicadoras binarias (0 y 1). Estos cortes no se traslapan con los datos: no existe ningún municipio en los rangos vacíos entre categorías.
6. **Resguardo y carga:** El resultado limpio (125 filas y 22 columnas) se exportó al archivo `02_base_limpia/pobreza_municipal_jalisco_2020_limpia.csv` y se cargó en la tabla `datos_limpios` de MySQL con llave primaria en `clave_municipio`.

Comparación Antes vs. Después de la limpieza

- **Total de filas:** 2,493 filas en `datos_originales` → 125 filas en `datos_limpios` (100% de los municipios de Jalisco).
- **Total de columnas:** 146 columnas en el Excel original → 22 columnas estructuradas en `datos_limpios`.
- **Valores vacíos (NaN / NULL):** 23 filas con valores vacíos por columna en el archivo

original → 0 valores vacíos en `datos_limpios`.

- **Textos no numéricos ("n.d."):** 3 municipios a nivel nacional → 0 en `datos_limpios`.

5 Encabezados de la base de datos (original y limpia)

A) Encabezados de la base de datos original (combinación de filas 4 y 5 del Excel original)

- `Col_01`: Clave de entidad
- `Col_02`: Entidad federativa
- `Col_03`: Clave de municipio
- `Col_04`: Municipio
- `Col_07`: Población 2020* (leer nota al final del cuadro)
- `Col_10`: Pobreza → Porcentaje 2020
- `Col_13`: Pobreza → Personas 2020
- `Col_16`: Pobreza → Carencias promedio 2020
- `Col_19`: Pobreza extrema → Porcentaje 2020
- `Col_58`: Rezago educativo → Porcentaje 2020
- `Col_67`: Carencia por acceso a los servicios de salud → Porcentaje 2020
- `Col_76`: Carencia por acceso a la seguridad social → Porcentaje 2020
- `Col_85`: Carencia por calidad y espacios de la vivienda → Porcentaje 2020
- `Col_94`: Carencia por acceso a los servicios básicos en la vivienda → Porcentaje 2020
- `Col_103`: Carencia por acceso a la alimentación → Porcentaje 2020
- `Col_130`: Población con ingreso inferior a la línea de pobreza por ingresos → Porcentaje 2020

B) Encabezados de la base de datos limpia (`datos_limpios`, 22 columnas)

Columna	Tipo	Descripción
`clave_municipio`	Texto (5)	Clave INEGI del municipio (llave primaria)
`clave_entidad`	Texto (2)	Clave de la entidad (14 = Jalisco)
`entidad_federativa`	Texto	Nombre de la entidad
`municipio`	Texto	Nombre del municipio
`poblacion_2020`	Entero	Población total 2020
`pobreza_porcentaje`	Decimal	**Variable objetivo** y: porcentaje de población en pobreza
`pobreza_personas`	Entero	Personas en pobreza
`carencias_promedio`	Decimal	Promedio de carencias sociales por persona
`pobreza_extrema_porcentaje`	Decimal	Porcentaje en pobreza extrema
`rezago_educativo_porcentaje`	Decimal	Porcentaje con rezago educativo
`carencia_salud_porcentaje`	Decimal	Porcentaje sin acceso a servicios de salud
`carencia_seguridad_social_porcentaje`	Decimal	Porcentaje sin acceso a seguridad social
`carencia_vivienda_porcentaje`	Decimal	Porcentaje con carencia de calidad y espacios de la vivienda
`carencia_servicios_basicos_porcentaje`	Decimal	Porcentaje sin acceso a servicios básicos en la vivienda
`carencia_alimentacion_porcentaje`	Decimal	Porcentaje sin acceso a la alimentación
`ingreso_inferior_lpi_porcentaje`	Decimal	Porcentaje con ingreso inferior a la línea de pobreza por ingresos
`tipo_municipio_poblacion`	Categórica	Clasificación por población (`Urbano_Grande`, `Semiurbano_Medio`, `Rural_Pequeno`)
`nivel_intensidad_carencias`	Categórica	Clasificación por promedio de carencias (`Intensidad_Alta`, `Intensidad_Moderada`, `Intensidad_Baja`)
`ind_urbano_grande`	Binaria (0/1)	Indicadora de `Urbano_Grande`
`ind_semiurbano_medio`	Binaria (0/1)	Indicadora de `Semiurbano_Medio`
`ind_intensidad_alta`	Binaria (0/1)	Indicadora de `Intensidad_Alta`
`ind_intensidad_moderada`	Binaria (0/1)	Indicadora de `Intensidad_Moderada`

Tabla 1: Diccionario de datos de la base limpia (`datos_limpios`).

6 Clasificación de la variable objetivo y y de las variables x

7.1 Variable objetivo (y)

La variable objetivo es `pobreza_porcentaje`: el porcentaje de la población de cada municipio que se encuentra en situación de pobreza en 2020. Es una **variable cuantitativa continua** medida en escala de razón (cero tiene significado y los valores están acotados entre 0 y 100). Es la variable que el modelo intenta predecir.

Variable	Media	Desv. est.	Mín.	Mediana	Máx.
Pobreza (%) = y	40.70	9.96	17.43	39.93	80.09
Rezago educativo (%)	23.24	6.49	9.42	23.53	43.13
Carencia de salud (%)	29.29	12.59	6.32	28.10	73.53
Carencia de seguridad social (%)	69.05	14.01	38.95	71.13	89.82
Carencia de vivienda (%)	8.69	7.13	1.88	7.04	57.62
Carencia de servicios básicos (%)	14.64	13.16	0.31	10.46	74.63
Carencia de alimentación (%)	13.72	6.32	2.10	13.56	48.11
Población 2020 (habitantes)	67,097	205,242	1,823	17,697	1,458,652

Tabla 2: Estadísticos descriptivos de la variable objetivo y de las variables cuantitativas predictoras (125 municipios de Jalisco).

7.2 Variables predictoras cuantitativas (x)

Las carencias sociales son **variables cuantitativas continuas** en escala de razón (porcentajes entre 0 y 100) y la población es una variable cuantitativa discreta. Se usaron como predictoras:

- `rezago_educativo_porcentaje`, `carencia_salud_porcentaje`, `carencia_seguridad_social_porcentaje`, `carencia_vivienda_porcentaje`, `carencia_servicios_basicos_porcentaje` y `carencia_alimentacion_porcentaje`.

7.3 Variables categóricas (x)

Las variables categóricas son **cualitativas** y no pueden entrar al modelo tal cual; se convierten en variables indicadoras (dummy) de 0 y 1.

- `tipo_municipio_poblacion` (**nominal**, 3 categorías): `Rural_Pequeno` (55 municipios), `Semiurbano_Medio` (59) y `Urbano_Grande` (11). Se usó `Rural_Pequeno` como categoría de referencia y se incluyeron las indicadoras `ind_semiurbano_medio` e `ind_urbano_grande`. Un municipio que es urbano tiene valor 1 en `ind_urbano_grande` y 0 en las demás.
- `nivel_intensidad_carencias` (**ordinal**, 3 categorías): `Intensidad_Baja` (34 municipios), `Intensidad_Moderada` (73) y `Intensidad_Alta` (18). Tiene orden natural, pero **no se incluyó en el modelo**: sus categorías se calculan a partir del promedio de carencias (`carencias_promedio`), así que el modelo sólo repetiría la misma información de las carencias y produciría colinealidad excesiva.

7.4 Variables descartadas como predictoras

Variable	Motivo de exclusión
`ingreso_inferior_lpi_porcentaje`	Correlación de 0.94 con la y: prácticamente es la misma variable. La pobreza se define como ingreso bajo y al menos una carencia, por lo que incluirla haría el modelo circular.
`pobreza_extrema_porcentaje`	Es una parte de la y (subconjunto de la población en pobreza).
`carencias_promedio`	Es un resumen de las carencias ya incluidas; además define `nivel_intensidad_carencias`.
`nivel_intensidad_carencias`	Derivada de `carencias_promedio` (ver 7.3).
`pobreza_personas`	Es el conteo de la misma y (en personas, no en porcentaje).

Tabla 3: Variables de la base limpia que no se usaron como predictoras y el motivo.

7 Primeras pruebas de Machine Learning: regresión lineal simple y múltiple

8.1 Diseño de las pruebas

Las pruebas se implementaron en Python con `scikit-learn` (ajuste del modelo y validación cruzada) y `statsmodels` (estimación de los errores estándar y los valores p, que `scikit-`

`learn` no reporta). Los modelos se evalúan con tres indicadores:

- R^2 **y** R^2 **ajustado**: proporción de la variación de la pobreza que explica el modelo. El ajustado penaliza agregar variables que no aportan.
- **Validación cruzada (CV)**: los datos se dividen en 5 bloques; el modelo se ajusta con 4 y se evalúa con el bloque restante, rotando los bloques. Se repitió 20 veces con distintas divisiones, para un total de 100 evaluaciones. Mide la capacidad de predecir municipios que el modelo no vio al ajustarse, por lo que es más confiable que el R^2 dentro de muestra.
- **RMSE y MAE**: error promedio de predicción en puntos porcentuales. El RMSE penaliza más los errores grandes; el MAE es el error absoluto medio.

Los errores estándar son robustos a la heterocedasticidad (HC3), porque la dispersión de los residuos no es constante entre municipios.

8.2 Prueba 1: Regresión lineal simple

Modelo:

$$\widehat{\text{pobreza}} = \beta_0 + \beta_1 \cdot \text{carencia_vivienda}$$

Término	**Coef.**	**Error est.**	**IC 95%**	**Valor** p
Intercepto (β_0)	33.32	1.14	[31.09, 35.55]	< 0.001
Carencia de vivienda (β_1)	0.85	0.12	[0.62, 1.08]	< 0.001

Tabla 4: Resultados de la regresión lineal simple (n = 125).

- $R^2 = 0.369$ (R^2 ajustado $= 0.364$). El modelo explica el 37% de la variación de la pobreza municipal.
- Validación cruzada: $R^2_{CV} = 0.265$ (desviación estándar de 0.199), RMSE = 8.03, MAE = 6.20 puntos porcentuales.
- Interpretación: por cada punto porcentual adicional de carencia de vivienda, la pobreza municipal aumenta en 0.85 puntos porcentuales, en promedio. El efecto es positivo y significativo, lo que confirma la dirección de la hipótesis simple.

8.3 Prueba 2: Regresión lineal múltiple

Modelo:

$$\begin{aligned}\widehat{\text{pobreza}} = \beta_0 &+ \beta_1 \cdot \text{rezago_educativo} + \beta_2 \cdot \text{salud} + \beta_3 \cdot \text{seguridad_social} + \beta_4 \cdot \text{vivienda} \\ &+ \beta_5 \cdot \text{servicios_básicos} + \beta_6 \cdot \text{alimentación} + \beta_7 \cdot \text{ind_semiurbano_medio} + \beta_8 \cdot \text{ind_urbano}\end{aligned}$$

Término	**Coef.**	**Error est.**	**Valor p**	**IC 95%**
Intercepto	4.74	4.91	0.334	[−4.88, 14.35]
Rezago educativo	0.16	0.14	0.233	[−0.10, 0.43]
Carencia de salud	0.02	0.08	0.800	[−0.13, 0.17]
Carencia de seguridad social	**0.33**	0.10	**0.0005**	[0.15, 0.52]
Carencia de vivienda	**0.53**	0.19	**0.005**	[0.16, 0.89]
Carencia de servicios básicos	−0.04	0.11	0.719	[−0.25, 0.17]
Carencia de alimentación	**0.35**	0.14	**0.014**	[0.07, 0.63]
Semiurbano_Medio (vs. Rural)	−0.75	1.73	0.663	[−4.14, 2.63]
Urbano_Grande (vs. Rural)	0.80	2.81	0.777	[−4.72, 6.31]

Tabla 5: Resultados de la regresión lineal múltiple (n = 125). Los valores en negrita son significativos al 5%.

- $R^2 = 0.625$ (R^2 ajustado $= 0.599$). El modelo explica el 60% de la variación de la pobreza municipal, ajustando por el número de variables.
- Validación cruzada: $R^2_{CV} = 0.501$ (desviación estándar de 0.181), RMSE = 6.59, MAE = 4.54 puntos porcentuales.
- Multicolinealidad: el VIF máximo es de 3.60 (vivienda), por debajo del umbral de 5 que se usa como referencia. No hay problema grave de colinealidad.
- Interpretación: controlando por las demás variables, las carencias con efecto significativo son seguridad social, vivienda y alimentación. Rezago educativo, salud y servicios básicos no son significativas una vez incluidas las demás. Las indicadoras de tipo de municipio tampoco resultan significativas, por lo que el tamaño de la población no añade explicación una vez controladas las carencias.

8.4 Comparación de modelos y verificación de hipótesis

Modelo	**Predictoras**	R^2	R^2 **ajustado**	R^2_{CV}	**RMSE CV**
Simple (vivienda)	1	0.369	0.364	0.265	8.03
Múltiple	8	0.625	0.599	0.501	6.59

Tabla 6: Comparación del modelo simple y del modelo múltiple.

- **Hipótesis simple: se cumple**. La carencia de vivienda tiene un efecto positivo y significativo ($\beta_1 = 0.85$, $p < 0.001$).

- **Hipótesis múltiple: se cumple**. El R^2 ajustado sube de 0.364 a 0.599, el R^2_{CV} sube de 0.265 a 0.501 y el RMSE de validación baja de 8.03 a 6.59 puntos porcentuales.

8.5 Limitaciones

- **Tamaño de muestra:** 125 municipios de un solo estado. Los coeficientes pueden cambiar al usar otros estados o el total nacional; en el análisis preliminar a nivel nacional, algunos efectos (como servicios básicos) fueron distintos.
- **Asociación, no causalidad:** el modelo mide qué carencias van acompañadas de mayor pobreza, no qué causa la pobreza.
- **Circularidad parcial:** la pobreza se define con carencias y con ingreso. Por eso el R^2 no mide qué tanto explican las carencias la pobreza en general, sino qué tanto explican la parte de pobreza asociada a las carencias disponibles.
- **Dependencia entre municipios:** los municipios comparten entidad y condiciones regionales, por lo que sus observaciones no son completamente independientes.
- **Predicción en municipios atípicos:** algunos municipios tienen un error grande (por ejemplo, Atenguillo y Mixtlán tienen una pobreza real mucho menor a la predicha), por lo que conviene revisarlos antes de usar el modelo para focalizar recursos.

Anexo A. Programa de limpieza (01_limpieza.py)

Reproduce la base limpia a partir del archivo original sin modificarlo. Se verificó que el resultado coincide valor por valor con `pobreza_municipal_jalisco_2020_limpia.csv`.

```
"""
01_limpieza.py
Limpia el Concentrado de indicadores de pobreza 2020 (CONEVAL, nivel municipal)
y genera la base limpia de Jalisco (125 municipios, 22 columnas).

Entrada : 01_base_original/Concentrado_indicadores_de_pobreza_2020.xlsx (sin modificar)
Salida  : 02_base_limpia/pobreza_municipal_jalisco_2020_limpia.csv
Ejecutar desde la carpeta raiz del proyecto: python 03_programas/01_limpieza.py
"""
from pathlib import Path
import pandas as pd
import numpy as np

ORIGEN = Path("01_base_original/Concentrado_indicadores_de_pobreza_2020.xlsx")
DESTINO = Path("02_base_limpia/pobreza_municipal_jalisco_2020_limpia.csv")

# 1. Lectura del libro sin encabezado (los encabezados estan repartidos en las filas 4 y 5)
raw = pd.read_excel(ORIGEN, sheet_name="Concentrado municipal", header=None, dtype=object)

# 2. Filtrado: solo filas de municipios (clave de municipio de 5 digitos).
#    Quedan fuera titulos, renglones vacios y notas metodologicas al final.
datos = raw.iloc[6:].copy()
es_municipio = datos[3].astype(str).str.fullmatch(r"\d{5}")
datos = datos[es_municipio]

# 3. Seleccion y renombrado de columnas (indices de columna del Excel original)
mapa = {
    1: "clave_entidad",
    2: "entidad_federativa",
    3: "clave_municipio",
    4: "municipio",
    7: "poblacion_2020",
    10: "pobreza_porcentaje",
    13: "pobreza_personas",
    16: "carencias_promedio",
    19: "pobreza_extrema_porcentaje",
    58: "rezago_educativo_porcentaje",
```

```
    67: "carencia_salud_porcentaje",
    76: "carencia_seguridad_social_porcentaje",
    85: "carencia_vivienda_porcentaje",
    94: "carencia_servicios_basicos_porcentaje",
    103: "carencia_alimentacion_porcentaje",
    130: "ingreso_inferior_lpi_porcentaje",
}
df = datos[list(mapa)].rename(columns=mapa)

# 4. Acotacion a Jalisco (antes de convertir a numero: hay "n.d." en otros estados)
df["clave_entidad"] = df["clave_entidad"].astype(str).str.zfill(2)
df = df[df["clave_entidad"] == "14"].reset_index(drop=True)

# 5. Conversion de tipos
df["clave_municipio"] = df["clave_municipio"].astype(str).str.zfill(5)
for c in df.columns.difference(["clave_entidad", "entidad_federativa", "clave_municipio",
     "municipio"]):
    df[c] = pd.to_numeric(df[c], errors="raise") # aqui fallaria si hubiera "n.d."
df["poblacion_2020"] = df["poblacion_2020"].astype(int)
df["pobreza_personas"] = df["pobreza_personas"].astype(int)
columnas_pct = [c for c in df.columns if c.endswith("_porcentaje") or c == "
    carencias_promedio"]
df[columnas_pct] = df[columnas_pct].round(2)

# 6. Variables categoricas derivadas (cortes observados en la base limpia)
#    tipo de municipio por poblacion: Rural < 15,000 <= Semiurbano < 100,000 <= Urbano
df["tipo_municipio_poblacion"] = np.select(
    [df["poblacion_2020"] < 15000, df["poblacion_2020"] < 100000],
    ["Rural_Pequeno", "Semiurbano_Medio"],
    default="Urbano_Grande",
)
#    nivel de intensidad por carencias promedio: Baja < 1.90 <= Moderada < 2.30 <= Alta
df["nivel_intensidad_carencias"] = np.select(
    [df["carencias_promedio"] < 1.90, df["carencias_promedio"] < 2.30],
    ["Intensidad_Baja", "Intensidad_Moderada"],
    default="Intensidad_Alta",
)
# 7. Indicadoras binarias (0/1)
df["ind_urbano_grande"] = (df["tipo_municipio_poblacion"] == "Urbano_Grande").astype(int)
df["ind_semiurbano_medio"] = (df["tipo_municipio_poblacion"] == "Semiurbano_Medio").
    astype(int)
df["ind_intensidad_alta"] = (df["nivel_intensidad_carencias"] == "Intensidad_Alta").
    astype(int)
df["ind_intensidad_moderada"] = (df["nivel_intensidad_carencias"] == "Intensidad_Moderada
```

```
    ").astype(int)

# 8. Controles de calidad
assert len(df) == 125, f"Se esperaban 125 municipios y hay {len(df)}"
assert df.isna().sum().sum() == 0, "Hay valores vacios"
assert df["clave_municipio"].is_unique, "Hay claves duplicadas"
assert df.filter(like="_porcentaje").apply(lambda s: s.between(0, 100)).all().all()

# 9. Exportacion (mismo orden de columnas que la base limpia entregada)
orden = ["clave_municipio", "clave_entidad", "entidad_federativa", "municipio", "
    poblacion_2020",
         "pobreza_porcentaje", "pobreza_personas", "carencias_promedio", "
             pobreza_extrema_porcentaje",
         "rezago_educativo_porcentaje", "carencia_salud_porcentaje", "
             carencia_seguridad_social_porcentaje",
         "carencia_vivienda_porcentaje", "carencia_servicios_basicos_porcentaje",
         "carencia_alimentacion_porcentaje", "ingreso_inferior_lpi_porcentaje",
         "tipo_municipio_poblacion", "nivel_intensidad_carencias",
         "ind_urbano_grande", "ind_semiurbano_medio", "ind_intensidad_alta", "
             ind_intensidad_moderada"]
df = df[orden]
DESTINO.parent.mkdir(parents=True, exist_ok=True)
df.to_csv(DESTINO, index=False, encoding="utf-8")
print(f"Base limpia: {df.shape[0]} filas x {df.shape[1]} columnas -> {DESTINO}")
```

Anexo B. Programa de modelos (02_modelos.py)

Ajusta la regresión lineal simple y múltiple, calcula la validación cruzada y el VIF, e imprime los resultados.

```
"""
02_modelos.py
Regresion lineal simple y multiple: porcentaje de pobreza municipal 2020 (Jalisco).

Y (variable objetivo) : pobreza_porcentaje
Simple              : pobreza ~ carencia_vivienda
Multiple            : pobreza ~ rezago educativo + salud + seguridad social + vivienda
                      + servicios basicos + alimentacion
                      + indicadoras de tipo de municipio (ref.: Rural_Pequeno)

Excluidas a proposito:
- ingreso_inferior_lpi_porcentaje -> correlacion 0.94 con la Y (casi la misma variable)
- nivel_intensidad_carencias  -> se define con carencias_promedio (circular)
- pobreza extrema / moderada / vulnerables / no pobre -> partes o complementos de la Y
Ejecutar desde la raiz del proyecto: python 03_programas/02_modelos.py
"""
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RepeatedKFold, cross_validate
from sklearn.metrics import r2_score

DATOS = "02_base_limpia/pobreza_municipal_jalisco_2020_limpia.csv"
Y = "pobreza_porcentaje"

CARENCIAS = [
    "rezago_educativo_porcentaje",
    "carencia_salud_porcentaje",
    "carencia_seguridad_social_porcentaje",
    "carencia_vivienda_porcentaje",
    "carencia_servicios_basicos_porcentaje",
    "carencia_alimentacion_porcentaje",
]
TIPO = ["ind_semiurbano_medio", "ind_urbano_grande"]

```

```
MODELOS = {
    "Simple": ["carencia_vivienda_porcentaje"],
    "Multiple": CARENCIAS + TIPO,
}

df = pd.read_csv(DATOS, dtype={"clave_municipio": str, "clave_entidad": str})
print(f"Municipios: {len(df)} | Y = {Y}\n")

cv = RepeatedKFold(n_splits=5, n_repeats=20, random_state=42)
resumen = []

for nombre, X_cols in MODELOS.items():
    X = df[X_cols].astype(float)
    y = df[Y].astype(float)
    Xc = sm.add_constant(X)

    # --- Inferencia (statsmodels, errores robustos HC3) ---
    ols = sm.OLS(y, Xc).fit(cov_type="HC3")
    print(f"================== Modelo {nombre} ==================")
    print(f"R2 = {ols.rsquared:.4f} | R2 ajustado = {ols.rsquared_adj:.4f} | "
          f"RMSE dentro de muestra = {np.sqrt(ols.mse_resid):.2f}")
    tabla = pd.DataFrame({
        "coef": ols.params, "error_HC3": ols.bse, "p_valor": ols.pvalues,
        "IC95_inf": ols.conf_int()[0], "IC95_sup": ols.conf_int()[1],
    }).round(4)
    print(tabla.to_string(), "\n")

    # --- Validacion cruzada (scikit-learn) ---
    modelo_sk = make_pipeline(StandardScaler(), LinearRegression())
    cvres = cross_validate(
        modelo_sk, X, y, cv=cv,
        scoring={"r2": "r2", "rmse": "neg_root_mean_squared_error", "mae": "
            neg_mean_absolute_error"},
    )
    print(f"Validacion cruzada 5x20: R2 = {cvres['test_r2'].mean():.4f} (sd {cvres['
        test_r2'].std():.3f}) | "
          f"RMSE = {-cvres['test_rmse'].mean():.2f} | MAE = {-cvres['test_mae'].mean():.2f
              }\n")

    # --- Colinealidad (solo en el multiple) ---
    if len(X_cols) > 1:
        vifs = pd.Series([variance_inflation_factor(Xc.values, i) for i in range(1, Xc.
            shape[1])], index=X_cols)
        print("VIF:\n" + vifs.round(2).to_string() + "\n")
```

```
80
81      resumen.append({
82          "modelo": nombre,
83          "k_predictoras": len(X_cols),
84          "R2": round(ols.rsquared, 4),
85          "R2_ajustado": round(ols.rsquared_adj, 4),
86          "R2_CV": round(cvres["test_r2"].mean(), 4),
87          "RMSE_CV": round(-cvres["test_rmse"].mean(), 2),
88      })
89
90  print("=================== Comparacion ===================")
91  print(pd.DataFrame(resumen).to_string(index=False))
```